Lab-Assignment 5.4

NAME: P.RANADHEER REDDY

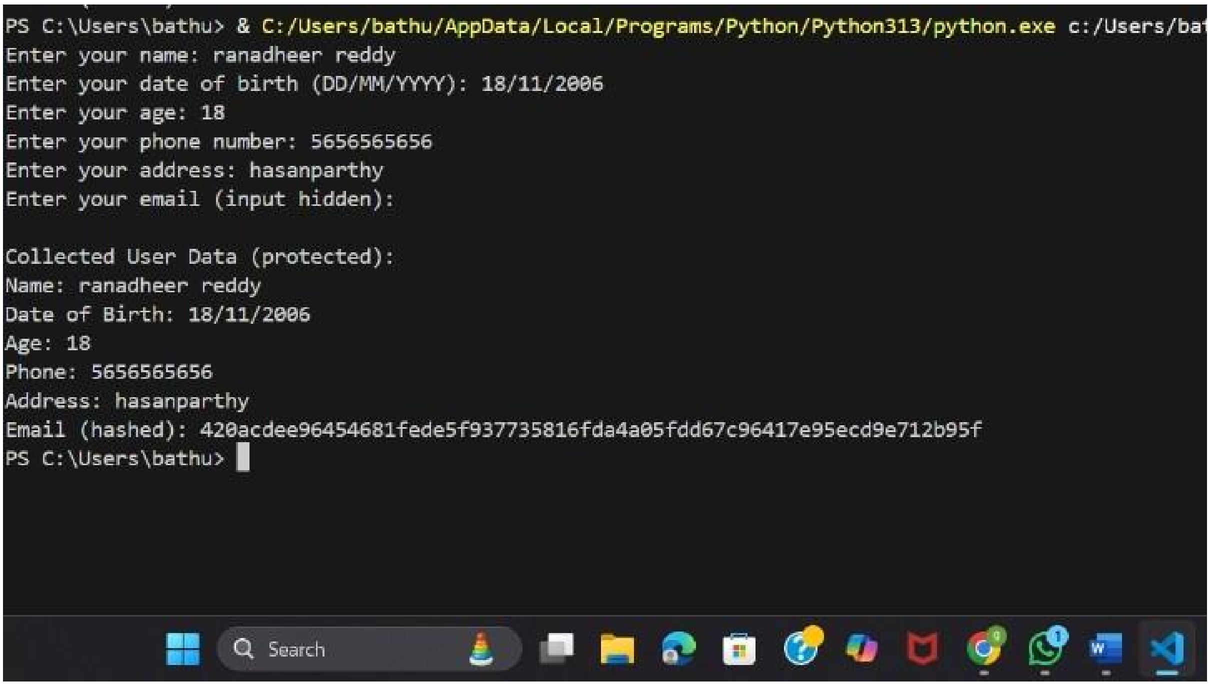
HT.NO: 2403A54098

BATCH: 18

Task-1:

PROMPT: Generate a python code that takes user personal details as input Eg(Name, Date of Birth, Age, Phone number, address, email) and the data should be protected and while entering the mail it should not be visible.

CODE & OUTPUT:



```
PS C:\Users\bathu> & C:/Users/bathu/AppData/Local/Programs/Python/Python313/python.exe c:/Users/bat
Enter your name: ranadheer reddy
Enter your date of birth (DD/MM/YYYY): 18/11/2006
Enter your age: 18
Enter your phone number: 5656565656
Enter your address: hasanparthy
Enter your email (input hidden):

Collected User Data (protected):
Name: ranadheer reddy
Date of Birth: 18/11/2006
Age: 18
Phone: 5656565656
Address: hasanparthy
Email (hashed): 420acdee96454681fede5f937735816fda4a05fdd67c96417e95ecd9e712b95f
PS C:\Users\bathu>
```

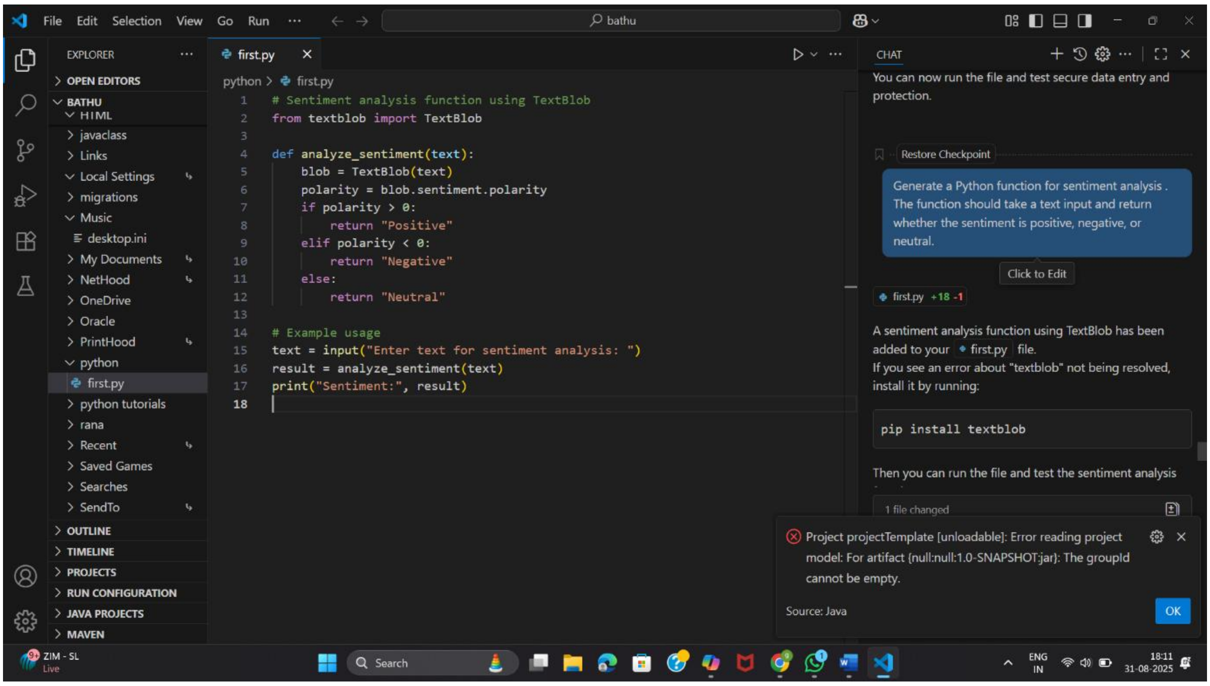

Task-2:

<u>PROMPT:</u>

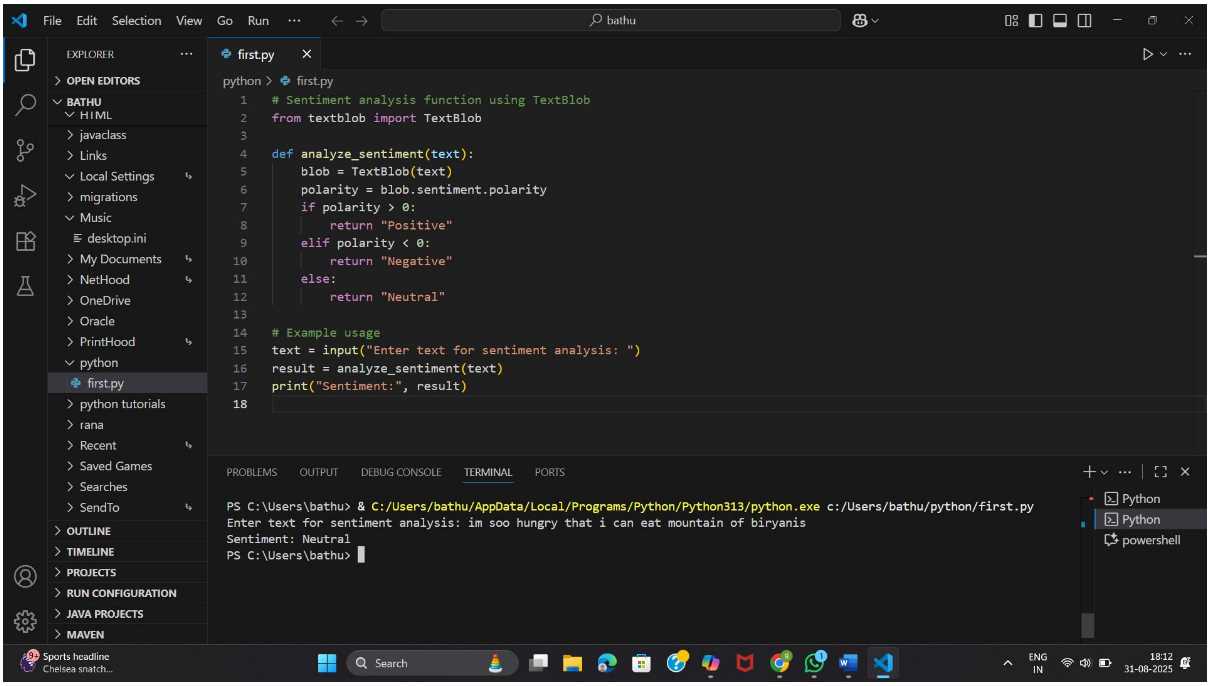
Generate a Python function for sentiment analysis .

The function should take a text input and return whether the sentiment is positive, negative, or neutral.

<u>CODE:</u>



Output:



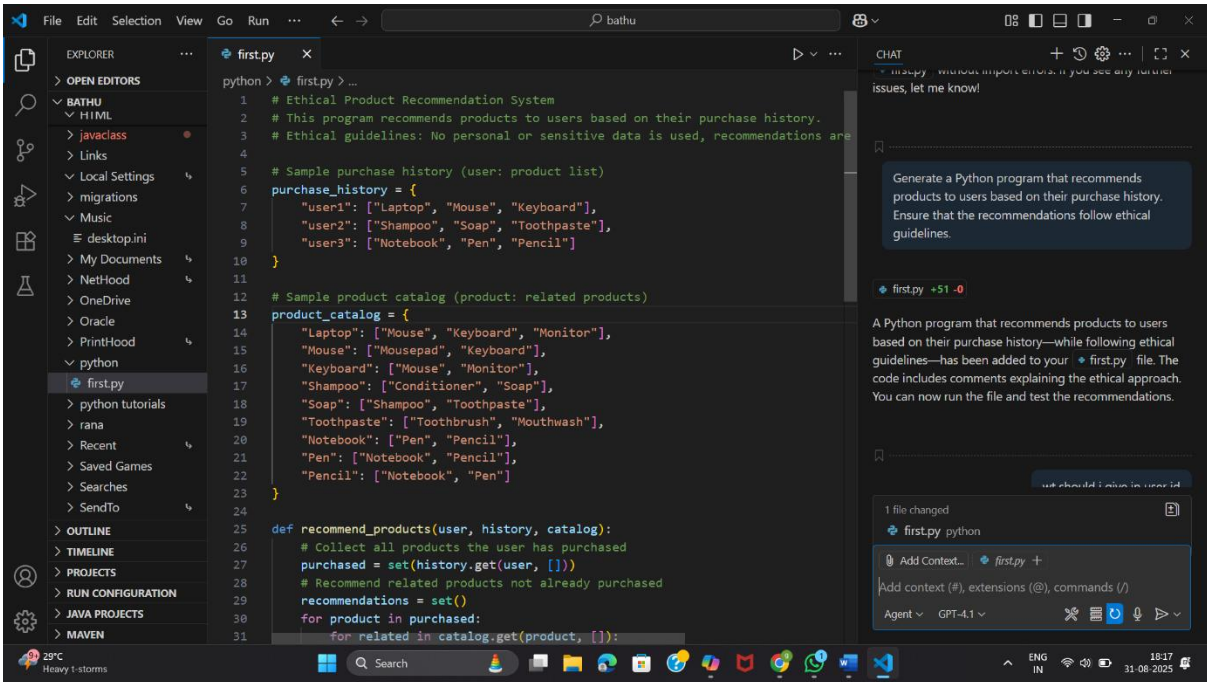
Task-3:

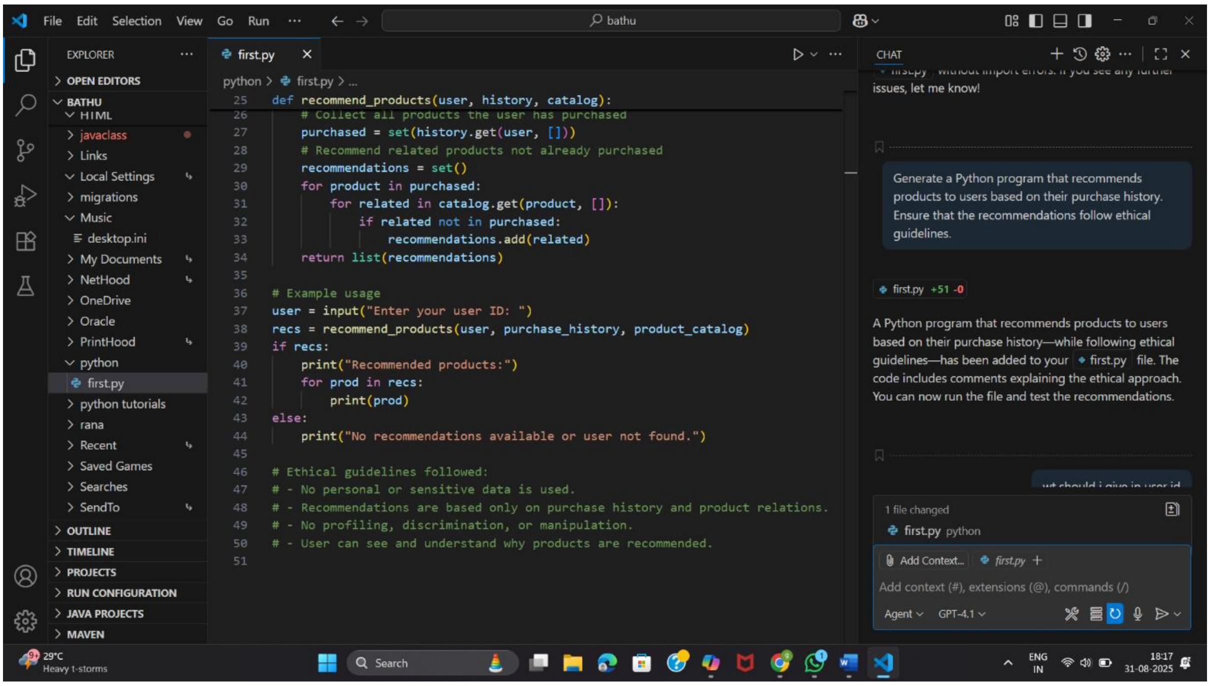
<u>PROMPT:</u>

Generate a Python program that recommends products to users based on their purchase history.

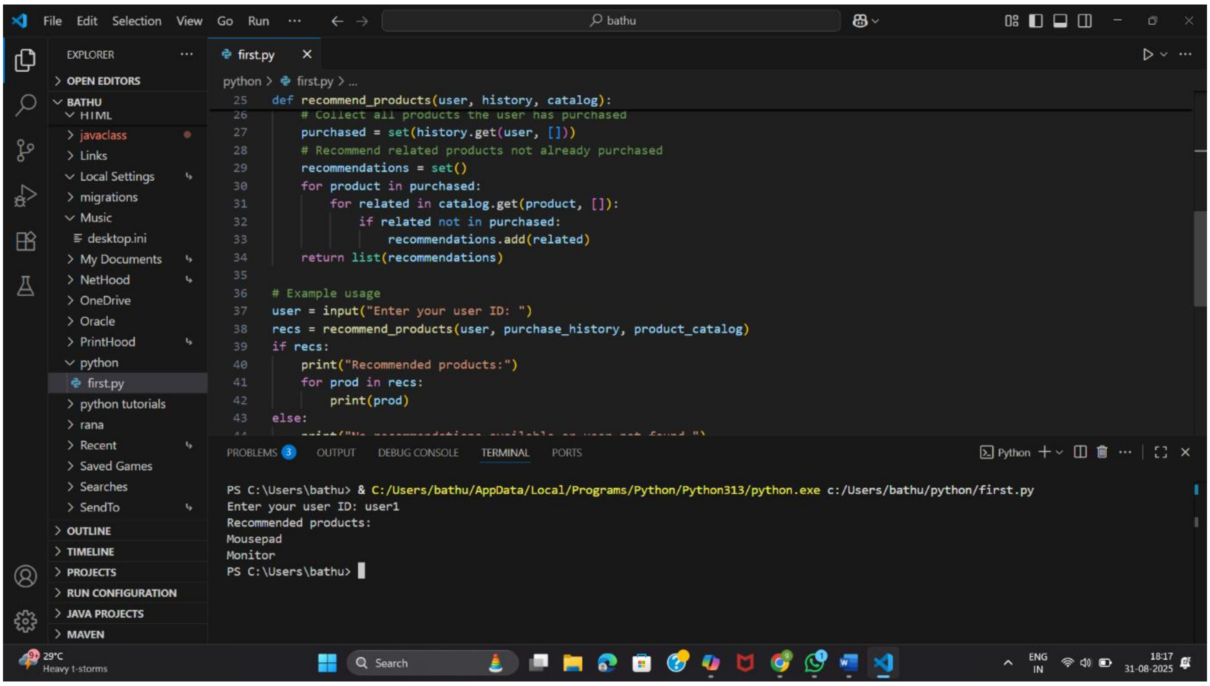
Ensure that the recommendations follow ethical guidelines.

CODE :-


first.py
# Ethical Product Recommendation System
# This program recommends products to users based on their purchase history.
# Ethical guidelines: No personal or sensitive data is used, recommendations are
# Sample purchase history (user: product list)
purchase_history = {
"user1": ["Laptop", "Mouse", "Keyboard"],
"user2": ["Shampoo", "Soap", "Toothpaste"],
"user3": ["Notebook", "Pen", "Pencil"]
}
# Sample product catalog (product: related products)
product_catalog = {
"Laptop": ["Mouse", "Keyboard", "Monitor"],
"Mouse": ["Mousepad", "Keyboard"],
"Keyboard": ["Mouse", "Monitor"],
"Shampoo": ["Conditioner", "Soap"],
"Soap": ["Shampoo", "Toothpaste"],
"Toothpaste": ["Toothbrush", "Mouthwash"],
"Notebook": ["Pen", "Pencil"],
"Pen": ["Notebook", "Pencil"],
"Pencil": ["Notebook", "Pen"]
}
def recommend_products(user, history, catalog):
# Collect all products the user has purchased
purchased = set(history.get(user, []))
# Recommend related products not already purchased
recommendations = set()
for product in purchased:
for related in catalog.get(product, []):
CHAT
Generate a Python program that recommends products to users based on their purchase history. Ensure that the recommendations follow ethical guidelines.
A Python program that recommends products to users based on their purchase history—while following ethical guidelines—has been added to your first.py file. The code includes comments explaining the ethical approach. You can now run the file and test the recommendations.


def recommend_products(user, history, catalog):
# Collect all products the user has purchased
purchased = set(history.get(user, []))
# Recommend related products not already purchased
recommendations = set()
for product in purchased:
for related in catalog.get(product, []):
if related not in purchased:
recommendations.add(related)
return list(recommendations)
# Example usage
user = input("Enter your user ID: ")
recs = recommend_products(user, purchase_history, product_catalog)
if recs:
print("Recommended products:")
for prod in recs:
print(prod)
else:
print("No recommendations available or user not found.")
# Ethical guidelines followed:
# - No personal or sensitive data is used.
# - Recommendations are based only on purchase history and product relations.
# - No profiling, discrimination, or manipulation.
# - User can see and understand why products are recommended.

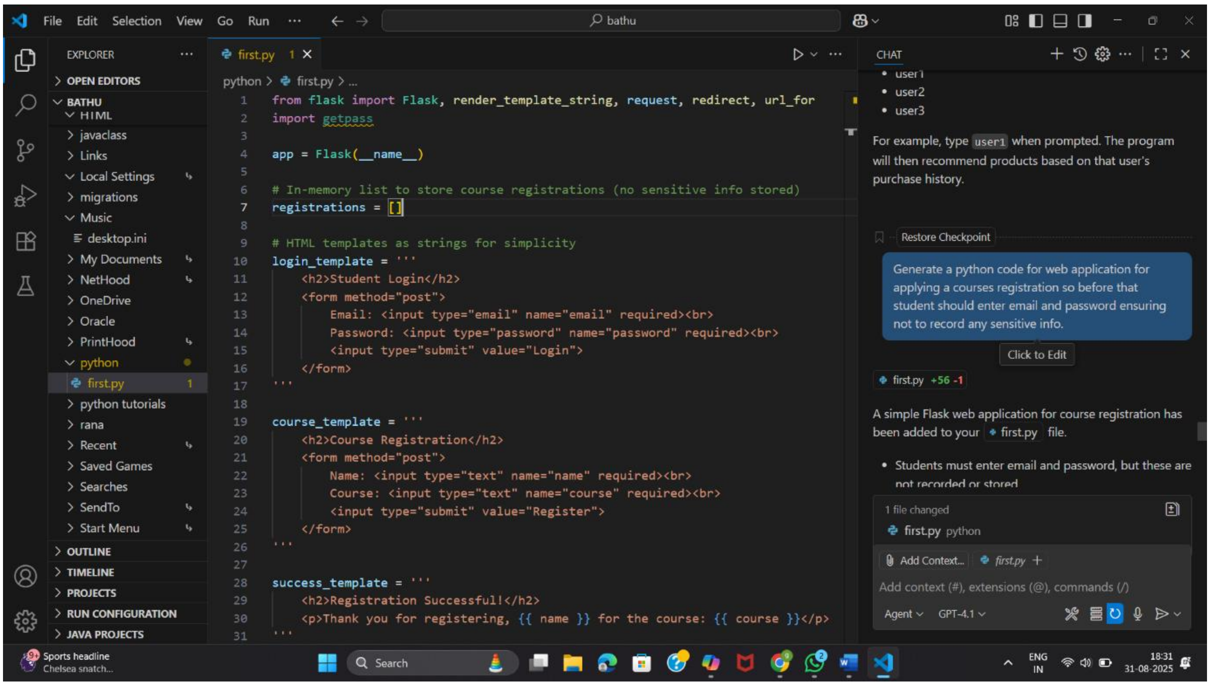
OUTPUT:

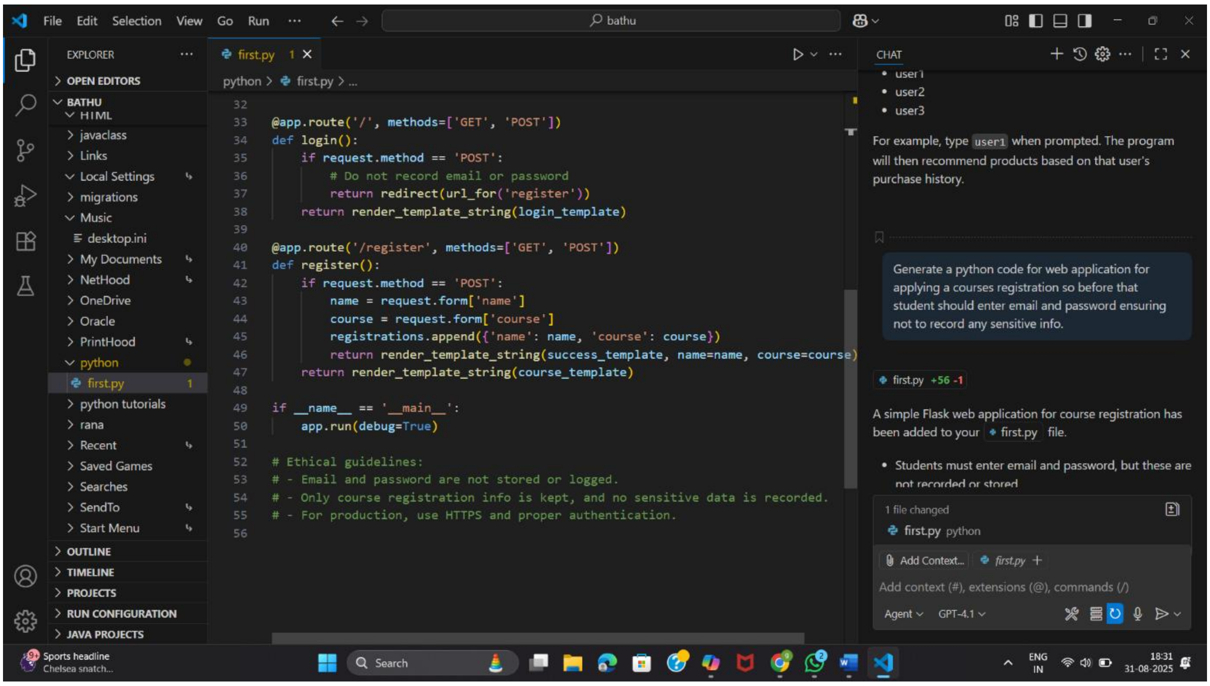


Task4:

prompt: Generate a python code for web application for applying a courses registration so before that student should enter email and password ensuring not to record any sensitive info.

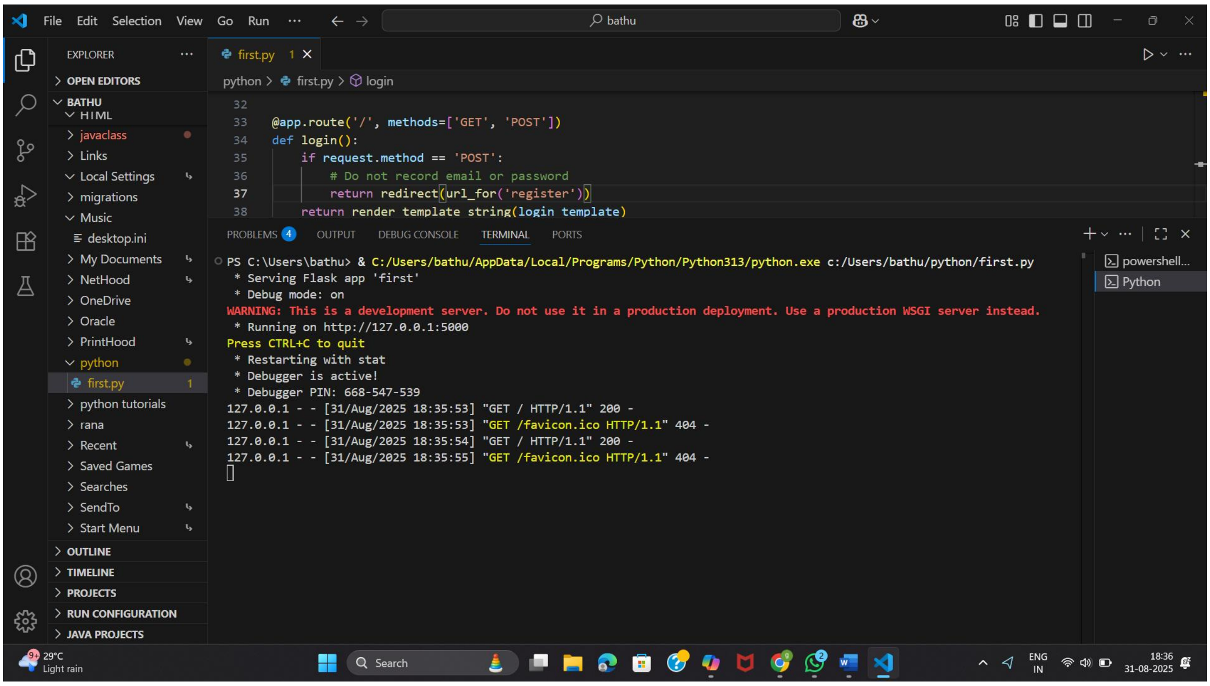
Code:



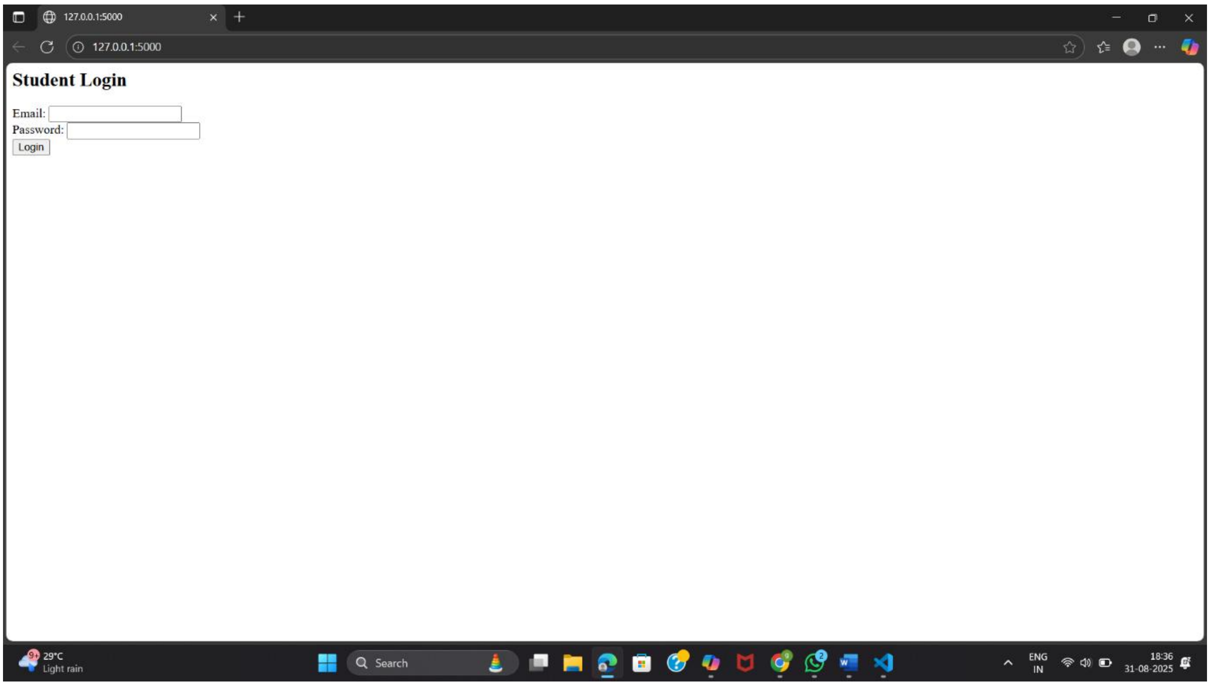


@app.route('/', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        # Do not record email or password
        return redirect(url_for('register'))
    return render_template_string(login_template)
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        name = request.form['name']
        course = request.form['course']
        registrations.append({'name': name, 'course': course})
        return render_template_string(success_template, name=name, course=course)
    return render_template_string(course_template)
if __name__ == '__main__':
    app.run(debug=True)
# Ethical guidelines:
# - Email and password are not stored or logged.
# - Only course registration info is kept, and no sensitive data is recorded.
# - For production, use HTTPS and proper authentication.
CHAT
For example, type user1 when prompted. The program will then recommend products based on that user's purchase history.
Generate a python code for web application for applying a courses registration so before that student should enter email and password ensuring not to record any sensitive info.
A simple Flask web application for course registration has been added to your first.py file.
Students must enter email and password, but these are not recorded or stored

Output:



PS C:\Users\bathu> & C:/Users/bathu/AppData/Local/Programs/Python/Python313/python.exe c:/Users/bathu/python/first.py
 * Serving Flask app 'first'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 668-547-539
127.0.0.1 - - [31/Aug/2025 18:35:53] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [31/Aug/2025 18:35:53] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [31/Aug/2025 18:35:54] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [31/Aug/2025 18:35:55] "GET /favicon.ico HTTP/1.1" 404 -



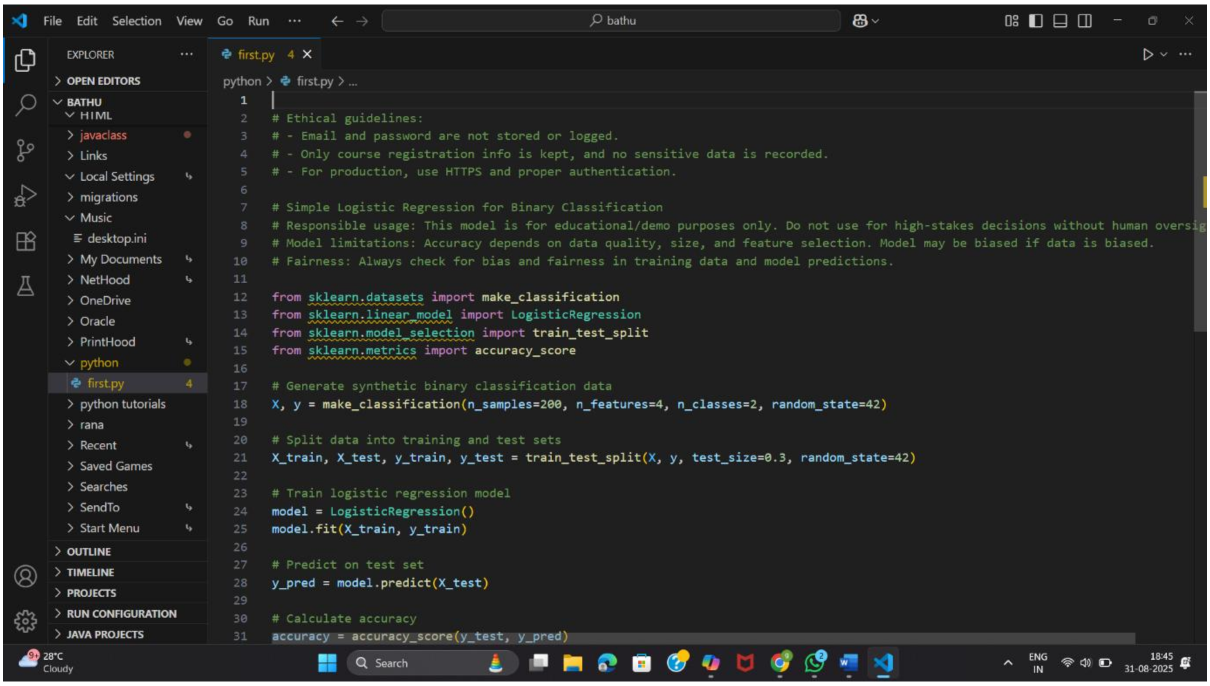
Task5:

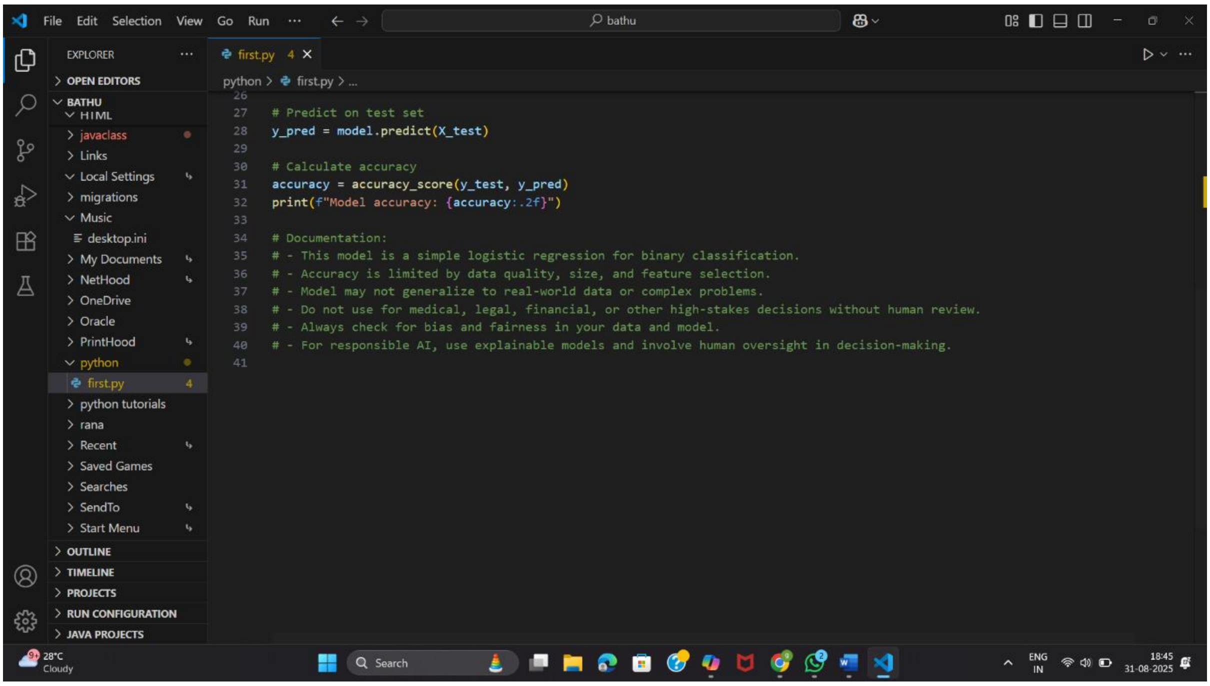
Prompt: Generate a Python program that trains a simple machine learning model (e.g., logistic regression for binary classification).

Add inline comments and documentation explaining responsible usage, model limitations, and fairness considerations.

Explain accuracy limits and note that the model should not be used for high-stakes decisions without human oversight.

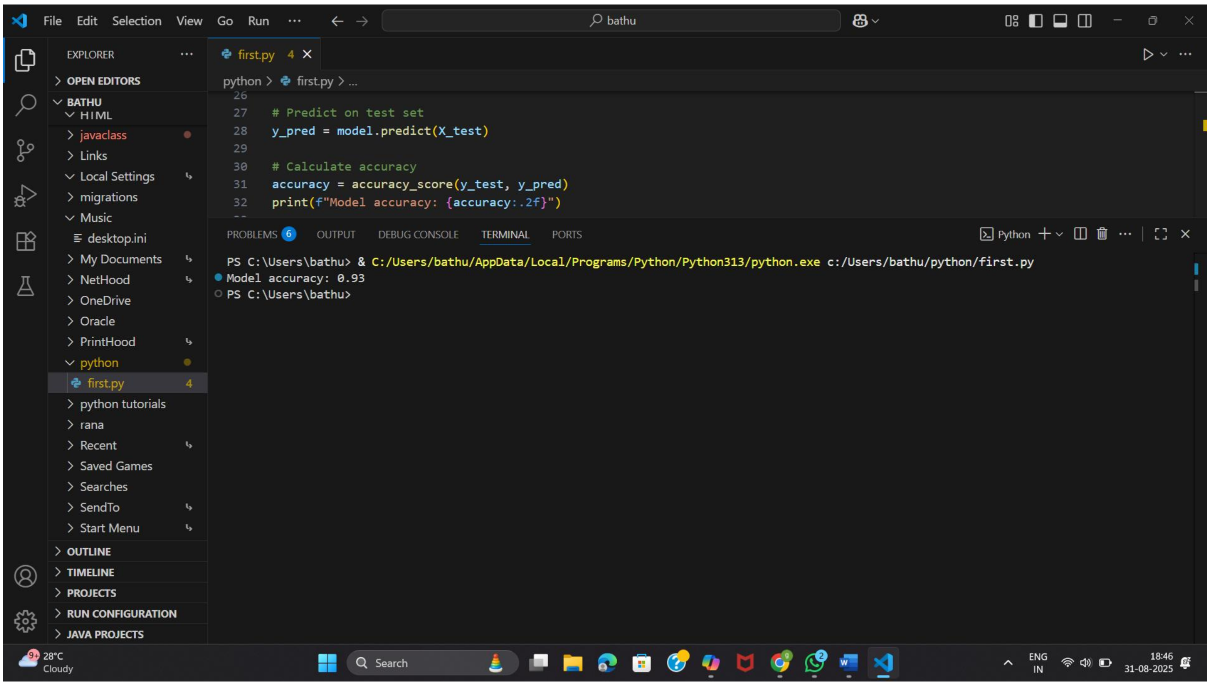
Code:





# Predict on test set
y_pred = model.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model accuracy: {accuracy:.2f}")
# Documentation:
# - This model is a simple logistic regression for binary classification.
# - Accuracy is limited by data quality, size, and feature selection.
# - Model may not generalize to real-world data or complex problems.
# - Do not use for medical, legal, financial, or other high-stakes decisions without human review.
# - Always check for bias and fairness in your data and model.
# - For responsible AI, use explainable models and involve human oversight in decision-making.

Output:



# Predict on test set
y_pred = model.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model accuracy: {accuracy:.2f}")
PS C:\Users\bathu> & C:/Users/bathu/AppData/Local/Programs/Python/Python313/python.exe c:/Users/bathu/python/first.py
Model accuracy: 0.93
PS C:\Users\bathu>